# Input Data Template

What to put in the file you upload — Labor Forecasting & Workforce Intelligence

The short version: **a date column and a number to forecast.** Everything else is optional and unlocks more of the application.

You do not need to match any schema, rename anything, or fill in a form first. Upload the file you already have; the application reads the columns, works out what they mean, and shows you the mapping to confirm before anything is trained.

> **Get a working example** In the app: **Data Quality** → **Upload Data** → **Download example file**. Or: `GET /api/datasets/template`
>
> That file is generated by the application itself, not stored alongside it, so it can never fall out of step with what the code accepts. Uploading it unchanged exercises every feature — grouping, drivers and the planned-vs- required comparison — which also makes it a quick way to confirm an install before you invest effort preparing a real export.

## The columns

The example file uses these names. **Yours can be called anything** — the names below only show the roles.

| Column | Role | What it is for |
|---|---|---|
| `business_date` | **Required** | The time axis. One row per period, per combination. |
| `hours_required` | **Required** | The value to forecast. The only column that must be numeric and present. |
| `region` | Splits into series | A dimension to forecast separately. |
| `department` | Splits into series | The second dimension. Region x department = 4 forecasts. |
| `hours_rostered` | Planned | What was already rostered. Unlocks Staffing Impact and gap recommendations. |
| `headcount_scheduled` | Planned | An alternative planned measure — pick whichever you roster by. |
| `occupied_beds` | Trains the model | A demand signal. Whatever drives your volume. |
| `admissions` | Trains the model | Another demand signal. |
| `meals_served` | Trains the model | Throughput measured alongside the target. |
| `transactions` | Trains the model | Till or order count. |
| `is_holiday` | Trains the model | Flag. Use `0`/`1`, not `Y`/`N`. |
| `is_promotion` | Trains the model | Flag for a campaign or event running that period. |

**The rule in one line:** the grouping columns decide *how many forecasts* you get; **every other column trains the models**.

## Splitting into several forecasts

This is the part most people misjudge. Grouping by two columns does **not** give you two forecasts — it gives you one per **combination**.

| Grouping columns | Series produced |
|---|---|
| none | 1 — the whole file as one forecast |
| `region` (2 values) | 2 |
| `region` + `department` (2 x 2) | **4** |
| 3 regions x 4 departments | 12 |

Pick the dimensions you genuinely want separate forecasts for, and no more. A column that is only a relabelling of the others — a `site` whose every value sits inside exactly one region-and-department pair — splits nothing further. It makes the series names longer and costs you a column you could have trained on instead, so it is dropped from the proposed grouping automatically.

Each series is trained **independently**: its own 13 models, its own cross-validation, its own champion. So the grouping you pick is also the main driver of how long training takes. Four series of a year's daily data take a few minutes; forty will take considerably longer.

**Only combinations that actually occur are created.** If Retail & Cafe exists in North Region but not South, you get three series, not four — an absent pairing would otherwise produce an empty forecast that fails at training with a confusing "not enough history".

The cap is **40 series per file**. Above that, the extra combinations are not created.

### What grouping buys you beyond separate numbers

Recommendations look **across** your series for spare capacity at the same moment. A shortfall in North Region / Retail & Cafe on a Tuesday morning gets matched against a surplus in South Region / Retail & Cafe at that same hour, and the action says to move it and names the source. With one ungrouped series there is nowhere to move anything from.

## What each role buys you

**Grouping columns** — described above.

**A planned column** is what Staffing Impact compares the forecast against. Without one that tab has a single curve and says so; with it you get the over/under-coverage breakdown and the gap recommendations.

It is deliberately **excluded from the model inputs**, and so is every other planned measure in the file. A roster is built in response to expected demand, so a model given it can predict demand from the plan made for that demand — which scores well and is worthless. If your file has both `hours_rostered` and `headcount_scheduled`, map one as planned; the other is kept out of the drivers automatically.

**Driver columns** are fed to the models as exogenous inputs. A driver that turns out to be flat once your data is aggregated is dropped automatically — it carries no information, and it can make the input matrix unusable for some models.

## Rules that actually matter

| File types | `.csv` `.tsv` `.txt` `.xlsx` `.xlsm` `.xls` |
|---|---|
| **Size** | up to 50 MB, up to 500,000 rows |

| File types | `.csv` `.tsv` `.txt` `.xlsx` `.xlsm` `.xls` |
|---|---|
| **Minimum history** | 20 periods per series, *after* resampling |
| **Maximum series** | 40 per file |

The app reports its own current limits under **What can I upload?** in the upload panel. If this document and the app disagree, the app is right — its numbers come from the code that enforces them.

- **Column order does not matter.** Nor does capitalisation of the names.
- **Gaps in the dates are fine.** The series is put on a regular grid before training, and gaps in the target are interpolated.
- **Several rows in the same period are fine.** They are combined by sum or average — you choose which, and it is prefilled sensibly (sum for counts and hours, average for rates and levels).
- **Extra columns you do not map are ignored**, not an error.

### Dates

Anything pandas can read: `2026-07-31`, `31/07/2026`, `2026-07-31 14:00:00`. ISO (`YYYY-MM-DD`) is safest if you have a choice.

**One timestamp column, not two.** If your export has a separate date and hour — `date` plus `hour_of_day` — the application uses the date alone, so 24 hourly rows collapse into one daily row and you are forecasting daily totals. To forecast hourly, combine them into a single column first:

```
business_date
2026-07-31 00:00:00
2026-07-31 01:00:00
```

### Flags

Use `0`/`1`. Text like `Y`/`N` is read as a category, not a flag.

## How the mapping is decided

On upload, the columns — names, types, a few sample values, how many distinct values each has — are sent to a language model, which returns which column plays which role. Your data rows are not sent; only the profile and a handful of samples.

Whatever comes back is **checked against the real columns** before you see it. A column that does not exist, a target that is not numeric, a date that does not parse, a "group" with one value per row, a driver that is really the target — each is rejected and the proposal falls back to detection from names and types.

The panel tells you which happened: *"columns mapped by AI"* or *"columns detected from names and types"*. With no API key configured, or if the call fails, it silently uses detection — an upload never fails because an external service was unavailable.

**You confirm the mapping either way.** Nothing is trained until you press Save mapping, and you can change any field first.